

■■■■ SIGNATURE (Sahi Signature)

AI Signature Forgery Detection System — Complete Architecture, Computer Vision Engineering, Mathematical Metrics & Interview Mastery Guide

Developed By: Ujjawal Groups	Audit Compliance: Ujjawal Groups Website Audit Verified
Developer: Ujjawal Sharma	Email: ujjawal77.0077@gmail.com
LinkedIn: linkedin.com/in/ujjawalsharma0804/	Security Model: Zero-Knowledge In-Memory RAM Validation

1. Executive Overview & Problem Statement

Signature forgery detection is a high-stakes biometric verification challenge widely present in legal, banking, and enterprise auditing domains. Traditional Deep Learning models (e.g., Convolutional Neural Networks) fail when applied to personalized reference verification because deep networks require thousands of samples per person to generalize without overfitting. In realistic verification scenarios, a system must determine whether a test signature is authentic using only **3 to 5 reference samples**.

■■■■ SIGNATURE solves this problem through a hybrid architecture combining **In-Memory OpenCV Computer Vision Preprocessing**, **9 Morphological & Statistical Feature Metric Extractors**, and a **Dual Machine Learning Verification Engine** (One-Class Anomaly Detection & Multi-Model Ensemble Classification).

2. End-to-End System Architecture & Pipeline Flow

The application architecture operates entirely in-memory to strictly adhere to enterprise zero-knowledge data privacy regulations. Uploaded image bytes or drawn canvas stroke Base64 arrays are passed directly through RAM streams without writing to disk.

Stage	Module Name	Input Payload	Core Processing Action	Output Result
1	Input Handler	File Upload / Canvas B64	Pillow BytesIO Stream Validation & MIME Check	RGB NumPy Array (RAM)
2	CV Preprocessor	RGB Pixel Matrix	Grayscale + Gaussian Filter + Otsu Binary + Crop	Normalized Bounding Box Ink
3	Feature Extractor	Cropped Ink Array	Morphological & Statistical Feature Calculation	9-Dimensional Feature Vector
4	ML Core	Reference + Test Vectors	One-Class Anomaly Distance OR SVM/RF Ensemble	Verdict & Match Confidence %
5	UI Visualizer	JSON Response Data	Radar Chart Rendering + Multi-Row CV Pipeline	Dynamic Verdict & Previews

3. Computer Vision (OpenCV & Scikit-Image) Preprocessing Pipeline

Raw signature photographs often suffer from variable lighting, paper texture noise, background shadows, and rotation variations. The computer vision pipeline standardizes all signature images prior to mathematical feature extraction using 5 sequential transformations:

- Stage 1 — Stream Validation & RGB Normalization:** Validates image format (PNG, JPG, JPEG, BMP), ensures file size < 5MB, verifies integrity using Pillow BytesIO, and converts color channels to standard 3-channel RGB matrix.
- Stage 2 — Luminance Grayscale Conversion:** Reduces 3-channel RGB to 1-channel intensity matrix using standard ITU-R 601-2 luminance weighting: $Y = 0.299R + 0.587G + 0.114B$.
- Stage 3 — Gaussian Noise Reduction:** Applies a 5x5 Gaussian Kernel blur ($\sigma = 1.0$) to smooth high-frequency scanner noise and digital stroke artifacts without destroying pen pressure contours.
- Stage 4 — Otsu Adaptive Binarization:** Automatically computes the optimal global threshold separating pen ink pixels from background paper by minimizing intra-class pixel variance:

Otsu Variance Formula: $\sigma_w^2(t) = q_1(t)\sigma_1^2(t) + q_2(t)\sigma_2^2(t)$

- Stage 5 — Bounding Box Contour Isolation:** Finds ink contours via Scikit-Image / OpenCV contour extraction, computes min/max ink coordinates, crops exact signature bounding box, and resizes to 128x256 normalized pixel matrix.

4. Mathematical Feature Engineering (9 Metric Extractors)

From the cropped binary ink array, 9 quantitative geometric, structural, and statistical metrics are calculated to form the signature's mathematical footprint vector $x \in \mathbb{R}^9$:

#	Feature Metric Name	Mathematical Definition & Formula	Biometric / Physical Significance
1	Aspect Ratio (ratio)	Ratio = Ink Area / (BBox Width * Height)	Measures overall ink density per bounding box area.
2	Vertical Mass Center (cent_y)	$Y_c = \frac{\sum (y * I(x,y))}{\sum I(x,y)}$	Vertical center of gravity of signature ink strokes.
3	Horizontal Mass Center (cent_x)	$X_c = \frac{\sum (x * I(x,y))}{\sum I(x,y)}$	Horizontal balance point along X-axis.
4	Contour Eccentricity	$E = \sqrt{1 - (b / a)^2}$	Measures elongation ratio of signature ellipse contour.
5	Convex Solidity	Solidity = Ink Area / Convex Hull Area	Ratio of pen ink area to total bounding convex envelope.
6	Horizontal Skewness (skew_x)	$Skew_x = \frac{E[(X - \mu_x)^3]}{\sigma_x^3}$	Measures horizontal pen stroke slant asymmetry.
7	Vertical Skewness (skew_y)	$Skew_y = \frac{E[(Y - \mu_y)^3]}{\sigma_y^3}$	Measures vertical stroke distribution asymmetry.
8	Horizontal Kurtosis (kurt_x)	$Kurt_x = \frac{E[(X - \mu_x)^4]}{\sigma_x^4}$	Detects peak stroke concentration & pen tremor noise.
9	Vertical Kurtosis (kurt_y)	$Kurt_y = \frac{E[(Y - \mu_y)^4]}{\sigma_y^4}$	Detects vertical stroke peakiness & flourishes.

5. Dual Machine Learning Verification Architecture

The system provides two operational ML modes tailored to different real-world availability of signature samples:

Option 1 — One-Class Anomaly Detection (Genuine Only Mode)

Used when only **3 or more genuine reference signatures** of a person are available (no forged samples available).

1. Computes mean profile vector μ_{gen} and standard deviation σ_{gen} across reference samples.
2. Calculates Z-Score matrix: $Z = |x_{test} - \mu_{gen}| / \sigma_{gen}$.
3. Computes Normalized Euclidean Distance and Cosine Similarity between test vector and genuine mean centroid.
4. **Dynamic Adaptive Thresholding:** Automatically adjusts verification threshold based on natural handwriting variance: $Threshold_{dynamic} = Threshold_{base} * (1.0 + \min(GenuineSpread, 1.5))$.

Option 2 — Full Training Ensemble (Genuine + Forged Mode) [RECOMMENDED]

Used when both **3+ genuine references** and **2+ forged samples** are provided. Achieves maximum precision by training a multi-model ensemble on-the-fly:

- **Model 1: Random Forest Classifier (100 Trees):** Handles non-linear feature interactions with balanced class weighting.
- **Model 2: Linear Support Vector Machine (SVM):** Fits optimal separating hyperplane in scaled feature space.
- **Model 3: Centroid Cosine Distance Classifier:** Compares cosine proximity to genuine centroid vs forged centroid.
- **Ensemble Decision Rule:** Majority voting (2/3 or 3/3 agreement) determines Genuine vs Forged verdict. Match confidence % is calculated from a weighted blend of model probability scores.

6. Frontend UI Innovations & Real-Time Inspection

- **Real-Time Live OpenCV Stroke Inspector:** As the user draws a signature on the HTML5 canvas, client-side Canvas ImageData processing instantly renders real-time Grayscale and Binarization previews below the canvas.
- **Smart Draw Reference Collection:** Users can draw multiple reference signatures directly on screen, saving snapshots into genuine/forged arrays with visual thumbnail cards and 1-click removal.
- **Multi-Row Computer Vision Pipeline Grid:** Visualizes stage-by-stage transformations (Raw, Grayscale, Binary, Cropped) for every signature image analyzed in color-coded rows (Genuine=Green, Forged=Red, Test=Gold).

7. Interview Mastery Cheat Sheet (Q&A; for Recruiters & Technical Interviews)

Q1: Why did you choose traditional CV + Feature Engineering over Deep Learning / CNNs?

Answer: Deep learning models require thousands of images per class to avoid severe overfitting. In biometric signature verification, users only provide 3 to 5 reference signatures. Feature engineering extracts invariant geometric traits (Aspect Ratio, Mass Center, Solidity, Kurtosis) that allow high-accuracy verification on very small sample sizes without requiring massive training datasets.

Q2: How does your system handle background noise, paper shadows, and lighting variations?

Answer: We apply a 5-stage OpenCV pipeline: Luminance conversion eliminates color variation, Gaussian blur removes high-frequency scanner noise, and Otsu Adaptive Binarization calculates the optimal intra-class variance threshold to cleanly separate ink strokes from background paper under any lighting.

Q3: What is the difference between your two Smart Verify modes?

Answer: Option 1 uses One-Class Anomaly Detection (Cosine + Euclidean + Z-score distance with dynamic thresholding) when only genuine signatures are available. Option 2 trains an on-the-fly ensemble of Random Forest, Linear SVM, and Centroid Cosine Distance when both genuine and forged samples are provided, achieving maximum accuracy by learning both decision boundaries.

Q4: How do you guarantee user privacy and data security?

Answer: All signature image files and Base64 canvas arrays are processed strictly in-memory (RAM) using Python BytesIO streams. No signature data or pixel arrays are ever saved to disk, database, or external logs. The server enforces strict Content Security Policy (CSP) headers and rate-limiting via Flask-Limiter.

Q5: How do you scale feature vectors for machine learning model input?

Answer: All 9 extracted feature metrics are normalized using StandardScaler (zero mean, unit variance) fitted on the reference dataset before feeding into Random Forest, Linear SVM, or calculating Euclidean feature distances.

Q6: How does the system handle subtle pen trembles or forgery attempts?

Answer: Pen trembles and hesitation marks alter high-frequency stroke distributions. Features like Horizontal and Vertical Kurtosis measure peak stroke concentration and kurtic noise, while Convex Solidity and Skewness detect irregular stroke envelope shapes typical of skilled forgeries.

8. Copyright & Brand Declaration

■■■ SIGNATURE — AI Signature Forgery Detection System

All Rights Reserved © 2026 Ujjawal Groups.

Integrated & Verified by Ujjawal Groups Website Audit Core.

Developer: Ujjawal Sharma | Email: ujjawal77.0077@gmail.com | LinkedIn: [linkedin.com/in/ujjawalsharma0804/](https://www.linkedin.com/in/ujjawalsharma0804/)